

RAG Pipeline — Modular Architecture Reference

Retrieval-Augmented Generation · LangChain · ChromaDB · SentenceTransformers · Modular src/ Structure

WHAT RAG retrieves relevant chunks from your own documents and passes them as context to an LLM, so answers come from your data, not training data.

WHY LLMs hallucinate on private or recent data. RAG grounds every answer in real retrieved content — no fine-tuning needed.

HOW Load files → split into chunks → embed → store in ChromaDB → embed query → fetch top-k → pass as LLM context → return answer.

■ Pipeline Flow



■ Module 1 — Data Loader (multi-format → LangChain Documents)

● 1 load_all_documents(data_dir)

src/data_loader.py

Loads **multiple file formats** (PDF, Word, Excel, text) from a directory and converts each into LangChain **Document** objects. **PyMuPDFLoader** returns one Document per PDF page.

Custom metadata — **file_name** and **file_type** — is attached to every Document so the source is traceable later. Returns a single flat list ready for chunking.

PyMuPDFLoader Multi-format Path.glob

```
from pathlib import Path
from typing import List, Any
from langchain_community.document_loaders import (
    PyMuPDFLoader, UnstructuredWordDocumentLoader,
    UnstructuredExcelLoader, TextLoader)

1 def load_all_documents(data_dir: str) -> List[Any]:
2     """Load all supported files from data_dir"""
3     documents = []
4     # Load PDF files and add to documents
5     for pdf in list(Path(data_dir).glob("./pdf_files/*.pdf")):
6         pdfFile = PyMuPDFLoader(pdf)
7         pdfLoad = pdfFile.load() # one Document per page
8         for doc in pdfLoad:
9             doc.metadata["file_name"] = pdf.name # track source file
10            doc.metadata["file_type"] = "pdf"
11            documents.extend(pdfLoad)
12            # Same pattern extends to .docx, .xlsx, .txt loaders
13    return documents
```

■ Module 2 — Embedding Manager (chunk + vectorize)

● 2 EmbeddingManager · get_chunks() + get_embedding()

src/embedding.py

Loads all-MiniLM-L6-v2 locally via **SentenceTransformer** in **__init__** — no API key needed. **get_chunks()** calls the loader, then splits Documents into **500-char chunks with 80-char overlap** using RecursiveCharacterTextSplitter.

get_embedding(text) converts a list of chunk strings into a 2D numpy array of **384-dimensional vectors** — one per chunk. Similar-meaning chunks get vectors pointing in nearly the same direction (used for cosine similarity later).

all-MiniLM-L6-v2 384-dim chunk=500/80

```
import numpy as np
from sentence_transformers import SentenceTransformer
from langchain_text_splitters import RecursiveCharacterTextSplitter
from src.data_loader import load_all_documents

1 class EmbeddingManager:
2     def __init__(self):
3         self.model = SentenceTransformer("all-MiniLM-L6-v2") # load once
4     def get_chunks(self, data_dir: str):
5         allDocs = load_all_documents(data_dir) # Module 1
6         doc_splitter = RecursiveCharacterTextSplitter(
7             chunk_size = 500, # max chars per chunk
8             chunk_overlap = 80, # shared chars between chunks
9             length_function = len
10        )
11        return doc_splitter.split_documents(
12            [doc for doc in allDocs])
13    def get_embedding(self, text: list[str]) -> np.ndarray:
14        return self.model.encode(text) # shape (N, 384)
```

■ Module 3 — Vector Store (ChromaDB persistent)

● 3

VectorDBManager · add_documents(documents, embeddings)

src/vector_store.py

Wraps ChromaDB PersistentClient — vectors are saved to disk so they survive restarts. `__init__` creates (or reuses) a collection named 'documents'. `add_documents()` pairs each Document with its embedding. Every chunk gets a **UUID-based id**, its original metadata, plus **doc_index** and **content_length** for filtering. All four lists are passed to `collection.add()` in one batch call.

PersistentClient

UUID ids

batch insert

```
import os, chromadb
from uuid import uuid4
```

```
1 class VectorDBManager:
2     def __init__(self):
3         os.makedirs("../data/vector-chromadb", exist_ok=True)
4         self.client = chromadb.PersistentClient(
5             path="../data/vector-chromadb")
6         self.collection = self.client.get_or_create_collection(
7             name="documents",
8             metadata={"description": "Vector DB for all documents"})
9
10    def add_documents(self, documents, embeddings):
11        idList, metadataList = [], []
12        documentList, embeddingList = [], []
13        for i, (document, embedding) in enumerate(
14            zip(documents, embeddings)):
15            id = f"doc_id_{i}_{uuid4().hex[:8]}" # unique ID
16            idList.append(id)
17            metadata = document.metadata
18            metadata["doc_index"] = i
19            metadata["content_length"] = len(document.page_content)
20            metadataList.append(metadata)
21            documentList.append(document.page_content)
22            embeddingList.append(embedding.tolist())
23        self.collection.add( # single batch insert
24            ids=idList, metadatas=metadataList,
25            documents=documentList, embeddings=embeddingList
26        )
```

■ Module 4 — RAG Retriever (semantic search)

● 4

RAGSearch · rag_retriever(query)

src/rag_search.py

Embeds the user **query** with the same model as Module 2, then calls `collection.query()` to fetch the **top 5** nearest chunks from ChromaDB by vector distance. Distance is converted to **similarity_score = 1 – distance** (1.0 = perfect match). Only chunks scoring **above 0.3** are kept — this filters out noisy, irrelevant matches before they reach the LLM.

top_k=5

score=1-distance

threshold>0.3

```
from src.vector_store import VectorDBManager
from src.embedding import EmbeddingManager
```

```
1 class RAGSearch:
2     def __init__(self, vectorStore: VectorDBManager,
3         embeddingManager: EmbeddingManager):
4         self.vectorStore = vectorStore
5         self.embeddingManager = embeddingManager
6
7     def rag_retriever(self, query: str):
8         queryEmbedding = self.embeddingManager.get_embedding(
9             [query])[0] # embed the query
10        gresult = self.vectorStore.collection.query(
11            query_embeddings=queryEmbedding,
12            n_results=5
13        ) # gresult = {ids, documents, metadatas, distances}
14        retrieved_docs = []
15        if gresult['documents'] and gresult['documents'][0]:
16            for id, doc, distance, metadata in zip(
17                gresult['ids'][0], gresult['documents'][0],
18                gresult['distances'][0], gresult['metadatas']
19            ):
20                similarity_score = 1 - distance
21                if similarity_score > 0.3: # filter weak matches
22                    retrieved_docs.append({
23                        'id': id, 'content': doc,
24                        'distance': distance, 'metadata': metadata,
25                        'similarity_score': similarity_score
26                    })
27        return retrieved_docs
```

```
def rag_simple(self, query):
    load_dotenv()
    llm = ChatOpenAI(model="gpt-4o-mini", api_key= os.getenv("OPENAI_API_KEY"))

    results = self.rag_retriever(query)
    context = "\n\n".join([doc["content"] for doc in results]) if results else ""

    message = [
        {
            "role": "system",
            "content": f"""You are an assistant, answer only from the supplied context""",
        },
        {
            "role": "user", "content": f"""
            context:{context}
            Question:{query}
            """
        }
    ]
    return llm.invoke(message).content
```

main.py

```
from src.embedding import EmbeddingManager
from src.vector_store import VectorDBManager
from src.search import RAGSearch
```

```
if __name__ == "__main__":
    emManager = EmbeddingManager()
    splitDocs = emManager.get_chunks("../data")
    embeddings = emManager.get_embedding([doc.page_content for doc in splitDocs])
```

```
vManager = VectorDBManager()
vManager.add_documents(splitDocs, embeddings)
```

```
ragSearch = RAGSearch(vManager, emManager)
result = ragSearch.rag_simple("Give details about option 1 leave")
print(result)
```

■ Quick Reference

Key Concepts

Document — page_content + metadata dict
Chunk — Sub-segment ≤500 chars
Embedding — 384-dim vector per chunk
similarity_score — 1 – distance; 1.0 = identical
threshold — Min score to keep a chunk (0.3)

Libraries

PyMuPDFLoader — Fast PDF loading
RecursiveCharTextSplitter — Smart chunking
SentenceTransformer — Local embeddings (CPU)
chromadb — Persistent local vector DB
uuid4 — Unique chunk IDs

Tips for Freshers

Overlap 15-20% of chunk_size avoids cutting context.
Persistent DB ChromaDB reuses embeddings on re-run.
threshold 0.3 filters irrelevant chunks before LLM.
Modular src/ keeps loader, embed, store, search separate.